

Proof of Stake on Sequentia: how it works today

A complete description of Sequentia's block production as implemented, current to 9 July 2026. It covers what a stake is, how a producer is elected, how a block is certified, and how producers are paid. It then documents the four consensus changes made this week: Taproot activation, staking-only vesting locks, stake delegation (staking pools), and payout policies. Every mechanism below is in the code and covered by tests; the closing section lists what is deliberately *not* solved. Written for a reader who knows Bitcoin but not this codebase.

1. The shape of the thing

Sequentia is a UTXO chain forked from Elements. It replaces Elements' fixed-federation signed block with a stake-weighted leader and a certifying committee. The block *signature* machinery is untouched: Elements already had "a block is valid if it satisfies a challenge script." Proof of Stake only changes **which key must sign** each block, never how the signature is checked.

Three properties frame everything else, and each explains a design decision that would otherwise look arbitrary.

Bitcoin anchoring is supreme. Every Sequentia block references a Bitcoin block header. Sequentia reorganises whenever Bitcoin reorganises. Anchoring outranks checkpoints and outranks finality. This is not a safety net bolted on; it is load-bearing in Proof of Stake specifically, because the per-block election seed is derived from the anchored Bitcoin hash. Bitcoin's proof of work is where Sequentia's randomness comes from, which is why no block producer can grind the election, and (as of this week) why none can bias a reward lottery either.

There is no inflation. All Sequence tokens are pre-mined; `genesis_subsidy = 0` and there is no coinbase issuance. A block producer's entire reward is the transaction fees in the block it produced. This is why the word "reward" below always means fees, and why the coinbase rules are so strict: the coinbase is the only place value can be claimed.

The Sequence token has no privileged standing except one. Fees are payable in any accepted issued asset, and no asset is a "native coin" in the user interface sense. The single exception is staking: only Sequence (ticker SEQ) can be staked, and consensus checks the asset identifier explicitly. Everything in this document is about that one exception.

2. What a stake actually is

This is the single most important idea in the document, and it is not what most people expect from a Proof of Stake chain.

A stake is an ordinary unspent output that matches a particular script pattern. It is never

moved, never locked in a module, never handed to anyone. Producing a block does not spend it, reference it, or sign over it. The chain counts the money sitting in it, and that is the whole mechanism.

The canonical staking script is a *bare* script, meaning it is stored unhashed in the output, so a validator can recognise a stake the moment the output is created without needing to see it spent:

```
<csv> OP_CHECKSEQUENCEVERIFY OP_DROP

    [ <bls_pubkey(48)> OP_DROP <bls_pop(96)> OP_DROP ]      # optional committee
registration

    [ <liquid_locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP ]    # optional vesting lock
(new, section 7)

<pubkey> OP_CHECKSIG
```

An output counts as stake, and contributes its full value as *weight*, if and only if three things hold (`StakeFromTxOut, src/pos.cpp`):

1. Its value and asset are **explicit**, that is, unblinded, and the asset is the policy asset (Sequence). A confidential amount cannot carry verifiable weight, because nobody can see how much it is.
2. Its `scriptPubKey` matches the template exactly. The parser is a closed template and rejects any trailing opcode.
3. Its relative timelock (`csv`) is at least the chain's minimum unbonding delay, compared as a wall-clock duration so that height-based and time-based encodings are judged on one axis.

The set of stakers and their weights is held in `StakeRegistry`, an in-memory index that is **a pure function of the UTXO set**. It is rebuilt by a full scan at startup and mirrored incrementally on every block connect and disconnect. That purity is what makes it reorganisation-safe for free: there is no separate state to roll back, only the UTXO set, which the node already rolls back correctly.

Two consequences follow, and both matter later:

- **Unbonding is simply the act of spending the staking output.** There is no unstaking transaction type. The `OP_CHECKSEQUENCEVERIFY` in the script enforces the delay directly, and the weight disappears when the output is spent.
- **A coin that cannot be spent can still stake.** Nothing about earning requires the ability to move. This is the fact that makes staking-only vesting periods expressible at all (section 7).

Weight is summed *per public key* across all of that key's staking outputs. The minimum-stake floor therefore applies to a key's total, not to any single output.

Mainnet parameters

parameter	value	meaning
total supply	400,000,000 SEQ	fully pre-mined, no issuance
minimum stake	40,000 SEQ	0.01% of supply. Below this a key is ignored by election, sortition, and the eligible-total denominator
slot interval	30 s	nominal block time; also the rank time gate
unbonding delay	43,200 blocks (~15 days)	minimum relative timelock for an output to count as stake
committee size	100 (configurable)	signers that certify each block
payout notice	2,880 blocks (~1 day)	delay before an announced payout policy binds (new, section 9)

3. How a producer is elected

Each block height has a seed, and the seed comes from Bitcoin:

```
seed = SHA256( parent block's Bitcoin anchor hash || height )
```

Because the anchor hash is the output of Bitcoin's proof of work, no Sequentia participant can choose it, and a producer cannot grind for a favourable one. Every node computes the same seed independently.

From the seed, each eligible staker gets a *weighted ticket*:

```
ticket = SHA256(seed || pubkey)  interpreted as a 256-bit integer  
value  = ticket / weight          # more weight, lower value, better rank
```

Sorting stakers by that value produces the slot's schedule. Rank 0 is the leader. A staker at rank r may only produce a block once $r \times 30$ seconds have elapsed since the parent, which is what allows the chain to route around a dead leader without any voting: the next-ranked producer simply becomes eligible.

The chain supports several signing modes, selected by chain flags. The current target design is the last one:

- **Public deterministic schedule.** The schedule above, with a script multisig committee, capped at 16 members by the script encoding.
- **Private VRF sortition.** Each producer proves, with a verifiable random function over the seed, that it was selected. Only the key holder can compute its own slot in advance, so an attacker

cannot know whom to attack. Under this mode any staker holding weight may produce a block when its slot arrives.

- **Aggregate committees.** MuSig2, and then BLS12-381, so that a hundred signatures compress into one and the committee can grow past the script multisig limit.
- **Public fixed-size committee with a bitfield certificate.** The committee is the schedule prefix restricted to stakers who have registered a BLS key. The certificate names its signers by a bitfield over that ordered prefix, and the aggregate signature verifies against the registered keys. Quorum is a strict majority, plus one at odd sizes, so that any two quorums overlap in at least two members. Two disjoint quorums therefore cannot certify rival blocks at the same height without at least two signers equivocating, which is attributable.

A staker's BLS committee key is carried *inside* its staking output, pushed and immediately dropped, so it does not affect the spend path. The registry learns it as a pure function of the UTXO set, exactly like the weight, and its proof of possession is verified once when the creating block connects.

The anti-freeze valve

If the committee cannot reach quorum, the chain would stall. Sequentia permits a below-quorum block once the parent chain's anchor has advanced by three Bitcoin blocks (roughly thirty minutes). Liveness is restored by Bitcoin's clock rather than by lowering the safety threshold in normal operation.

4. How a producer proves it controls the stake

There is no separate proof. The chain checks two things:

1. The staking output exists, unspent, so its weight is in the registry.
2. The block's elected leader key *is* the public key embedded in that staking script.

The proof of control is the block signature itself. The per-block challenge is `<leader> OP_CHECKSIG`, so only the holder of the corresponding private key can produce a valid block. The chain never asks anyone to sign over the coins, and never inspects the input outpoints of the producer.

This has an uncomfortable corollary, which was true until this week and is the reason section 8 exists: **the key that signs blocks is the same key that can spend the staked coins.** To earn, you had to keep the key that controls your money on an internet-facing server.

5. How a producer is paid

There is no subsidy, so the reward is exactly the fees in the block. Consensus requires that *every* coinbase output carrying value pays the elected leader, specifically a pay-to-witness-public-key-hash of the leader's key. Zero-value outputs (commitments, the witness commitment) are exempt. A block whose coinbase pays anyone else is rejected with `bad-coinbase-not-leader`.

Two details worth internalising, because they explain later sections. Rewards land in a **separate output**, never inside the staking output, so staking income is immediately spendable even when the

stake itself is frozen. And because the rule is "pay the leader, and only the leader," a pool operator producing blocks with borrowed weight would collect one hundred percent of the reward. That is the problem section 9 addresses.

6. Change one: Taproot now activates

Commit `bb1fb61f8`.

The Sequentia mainnet chain parameters carried Bitcoin's Taproot deployment window verbatim: start time 1619222400, timeout 1628640000, minimum activation height 709,632. Under Elements' `elements_mode`, those first two fields are interpreted as **block heights, not timestamps**. So the deployment was scheduled to begin signalling at block 1.6 billion, roughly 1,540 years out at thirty-second blocks. Taproot could never have activated.

Two consequences, the first serious:

- `SCRIPT_VERIFY_TAPROOT` is gated on that deployment. While Taproot is inactive, a witness version 1 output is an unencumbered **anyone-can-spend**. This was a funds-loss bug that would have shipped at genesis.
- Tapscript gates the Elements introspection opcodes, on which every covenant we ship depends, including SeqOB's passive order book.

Taproot is now always-active from genesis on the mainnet chain, matching testnet, regtest, and the buried Bitcoin soft forks above it. A regression test asserts this for every Sequentia network and fails against the old constants.

7. Change two: staking-only vesting periods

Commit `86257383a`. Test: `feature_pos_stake_vesting.py`.

The mainnet distribution gives each allocation a cliff, a linear vest, and a **staking-only period**: tokens unlock for staking first, and only become liquid, meaning sellable or transferable, some months later. Terms differ per allocation.

The mechanism is coherent only because of the fact established in section 2: a stake never moves. A coin that literally cannot be spent still counts, still produces blocks, and still earns fees. Non-transferability is a consequence of non-spendability, so no covenant is involved.

The staking script therefore gained an optional **absolute** timelock:

```
<csv> OP_CSV OP_DROP [ <bls> ... ] <liquid_locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP  
<pubkey> OP_CHECKSIG
```

`StakeFromTxOut` needed no change at all: weight never looked at spendability. That is the elegance of it. The output stakes throughout, and cannot be sold until the locktime.

A relative timelock could not have served. Relative locks are sixteen-bit, so the time-based encoding tops out at 65535×512 seconds, which is 388 days; the height-based encoding at thirty-second slots tops out at under 23 days. A two-year staking-only period is simply not expressible as a relative lock. Absolute locks are thirty-two-bit and reach any calendar date. Use unix-time locktimes rather than heights, because heights drift against a calendar over a multi-year schedule.

Two implementation details that would otherwise have been silent bugs. The locktime is distinguished from the staking public key by *one-token lookahead* (it is a locktime only if `OP_CHECKLOCKTIMEVERIFY` follows it), not by push size. And the value is decoded at full sixty-four-bit width, because `CScriptNum::getint()` saturates at `INT_MAX` and would have corrupted any locktime past 2038.

Enforcing the cliff, and what is not enforced

During the cliff a tranche should hold *no* stake weight. Achieve this by not making it a staking output yet: hold it in a plain `<cliff> OP_CLTV OP_DROP <key> OP_CHECKSIG` output, then spend it into the staking output when the cliff ends. Note honestly that nothing *forces* that spend to go into the staking output rather than to an exchange. Closing that requires the cliff output to be a tapscript covenant asserting that output zero pays exactly the tranche value to the literal staking script. Every field of that script is a constant known when the tranche is created, so this is a hardcoded hash comparison rather than a recursive covenant. It is now possible only because Taproot activates (section 6).

8. Change three: delegation, and non-custodial staking pools

Commit `d363c67dd`. Test: `feature_pos_delegation.py`.

Section 4 ended on the hazard: one key both signs blocks and spends the coins. Section 7 made it worse, because a staking output frozen for years cannot be respent, so any signer named *inside* it could never be replaced. A compromised operator key would be irrevocable for the entire lock.

The fix is an indirection. A staker (the **controller**) lends its weight to a **signer** by funding a small, separate record:

```
<"SEQDEL"> OP_DROP <signer> OP_DROP <controller> OP_CHECKSIG
```

While that record is unspent, the controller's weight counts for the signer, and the signer is the key that must produce and sign blocks. The record is spendable by the **controller alone**. The staking script itself did not change; its existing public key is now understood as the cold, coin-controlling key.

This buys three things at once.

- **Custody.** The signer does not appear in the staking script's `OP_CHECKSIG`, so it can never spend the stake. Not by promise, but because it does not hold the key. The coins never move.
- **A cure for the hot key.** Delegate to your own online key and keep the coin-controlling key offline. If the block-producing server is compromised, the attacker can sign blocks and nothing more.
- **Rotation under a vesting lock.** Re-pointing or reclaiming spends only the tiny record, never

the staking output. It works even while the stake is frozen for years.

Delegation resolution is **one hop, never chained**: if A delegates to B and B delegates to C, A's weight goes to B and B's own weight goes to C. Chasing chains would admit cycles. A cycle in the graph terminates rather than hanging.

Two hazards the implementation had to handle

Ordering. The block-apply path adds created outputs before subtracting spent ones. A rotation spends the old record and creates the new one in a single transaction, so a naive removal would have erased the freshly installed record. Removal is therefore conditional: it erases only if the record is still the one in force. The revert path is the exact inverse, restoring spent records before removing created ones.

Ambiguity. Two unspent records for one controller would make the signer, and therefore the leader election, ambiguous and node-dependent. That is a chain split. A consensus rule forbids it: `bad-delegation-conflict` for two in one block, `bad-delegation-exists` for one created while another is live.

The functional test exercises the properties directly. Under VRF sortition any staker holding weight may produce a block, so "who holds the weight" is observable as "who can produce a block": before delegating the owner can produce and the pool cannot, and afterwards the pool can and the owner cannot. The test also has the pool attempt to steal a delegator's stake, which fails on the controller's `OP_CHECKSIG`; rotates pools while the stake is vesting-locked, leaving the staking output untouched; and survives a restart and a reorganisation.

9. Change four: payout policies

Commits `a9a6a6a2f, 3ba5fb3ca`. Test: `feature_pos_payout.py`.

Delegation protects the coins but not the rewards. Under the rule in section 5, an operator producing blocks with borrowed weight collects all of it and is trusted to pay delegators back. A producer may now **commit** to a payout policy, announced on-chain in a record spendable by itself:

```
<"SEQPAY"> OP_DROP <activation> OP_DROP <mode> OP_DROP <param> OP_DROP <signer>  
OP_CHECKSIG
```

Both modes ship, and the market decides which it wants.

Direct mode

The coinbase must pay a committed script. The operator cannot silently redirect the reward. Be precise about what this does and does not guarantee: **the chain does not check that the destination is fair to anyone**. An operator may commit to its own address and keep everything. Direct mode is opt-in and trust-minimising, not trustless. What it provides is a public, immutable-under-notice commitment that a delegator can read before delegating.

Lottery mode

The coinbase must pay **one participant**, drawn from everyone whose weight counts for this signer, weighted by that weight. The draw is derived from the block's election seed, which comes from Bitcoin's proof of work, so no operator can bias it. Every node computes the same winner; a block paying anyone else is rejected.

Over many blocks each delegator earns its exact proportional share, enforced by consensus, with no per-delegator accounting and no epoch reward subsystem. The operator keeps a commission in basis points, agreed in the open. At zero commission it still earns on its own stake, as a participant like any other.

The cost is real and should not be glossed. **Lottery mode does not smooth income.** A delegator holding one percent of a pool does not receive one percent of every block; it receives all of one block in a hundred. The average is exactly right, the variance is not reduced, and variance reduction is one of the two reasons pools exist. The other reason survives: holders below the 40,000 SEQ floor cannot stake alone at all, and pooling lets them participate.

The notice period, which is what makes auditing meaningful

A policy may not bind until its `activation` height, which consensus requires to be at least the notice period (2,880 blocks, about a day) beyond the block that announces it. Announcing does not cancel an earlier policy: the policy in force at a height is the announced policy with the greatest activation at or below it, so a pending policy and the one it will replace coexist unambiguously through the notice window.

The reasoning is worth stating plainly, because without it the whole design is theatre. A delegator can leave a pool **instantly and unilaterally**, without the operator's consent, without an unbonding wait, and without the coins moving. Auditing a pool before delegating is therefore only meaningful if what you audited cannot silently change underneath you. Without a notice period an operator could accept your delegation, redirect the rewards on the very next block, and collect. With it, a hostile change is visible on-chain for a day before it pays a single coin, and you can always exit first.

This bounds the loss to zero for a delegator who is watching. It does nothing for one who is not. The mechanism makes the change visible; wallets must watch delegated pools and warn, or the guarantee degrades back to trust for anyone not looking. That is a wallet obligation, not a consensus one, and it is currently unbuilt.

The single seam

All of this lives in one function, `PosRequiredCoinbaseScript(leader, height, seed)`. The producer builds its coinbase from it and `ConnectBlock` enforces it, so the two cannot disagree. With no policy announced it returns the leader's own script, which is exactly the behaviour that existed before the feature. Note also that the notice period is consensus-critical and is pinned on every fixed chain; only the throwaway regtest parameters read it from a command-line flag, which is how the tests shorten it.

10. Consensus rejections added this week

code	meaning
bad-delegation-conflict	two delegation records for one controller in a single block
bad-delegation-exists	a delegation record created while the controller already has one unspent
bad-payout-notice	a payout policy that would bind before its notice period elapses
bad-payout-conflict	two payout records for one signer at one activation height, in a block
bad-payout-exists	a payout record duplicating an unspent record's activation height
bad-coinbase-not-leader	(existing, now broader) the coinbase does not pay the required payee

Remote procedure calls

call	purpose
getstakescript	build a staking output, optionally with a BLS registration and a vesting <code>liquid_locktime</code>
registerstake	fund one from the wallet
getstakerinfo	live weights, by signer
getdelegationscript	build a delegation record
getdelegationinfo	the live controller to signer map
getpayoutscript	announce a direct or lottery payout policy
getpayoutinfo	the audit surface: what each producer has committed to, and whether it is yet in force

11. What is deliberately not solved

Stating these plainly is more useful than a summary.

- **Direct mode does not enforce fairness.** It prevents a silent redirect. It does not, and cannot, check that an operator's committed destination shares anything with its delegators. The protection is exit plus notice plus reputation, not mathematics.
- **Nobody is watching for the delegator.** The notice period is only as good as the monitoring in front of it. No wallet currently warns on a pending payout change.
- **Lottery mode does not reduce variance.** It gives correct expected income with lumpy

realisation. The smoothed, trustless option is per-delegator reward accounting across epochs, which is a large subsystem and is not built.

- **A pool operator with no stake of its own cannot join the committee.** The BLS committee key rides inside a *staking* output. An operator that has never staked has nowhere to put it, and must fund a small staking output of its own to register.
- **Signing keys cannot be rotated inside a vesting lock without delegation.** Delegation solves this, and is the reason the signer lives outside the staking output. A staker who does not delegate is still exposed.
- **The staking cliff is not covenant-enforced yet.** See section 7.
- **Pre-existing, unrelated:** the legacy MuSig2 block template path (`getposblocktemplate`) does not pay the leader, and `feature_pos_distributed_committee.py` fails on the current branch. Both predate this week's work and were verified to fail before it.

12. In one paragraph

A stake on Sequentia is a coin sitting still in a recognisable output. Bitcoin's proof of work supplies the randomness that elects who produces the next block, weighted by how much is sitting there, and a committee of the top-ranked stakers certifies it with one aggregated signature. Because the coin never moves in order to earn, a coin that is forbidden to move can still earn: that is a staking-only vesting period. Because the block-signing right is named in a separate small output rather than inside the coin, that right can be lent to a pool, revoked instantly, and rotated even while the coin is frozen for years, and the pool can never touch the coin. And because the block reward's destination is checked against a policy the producer committed to a day in advance, a delegator can read what a pool promises, watch it, and leave before any change to it takes effect, or choose a pool that has handed the decision to Bitcoin's dice instead.

Sequentia is developed by Concatena Labs. The network is Sequentia; the token is called Sequence, ticker SEQ. Code references are to `src/pos.h`, `src/pos.cpp`, `src/validation.cpp` and `src/chainparams.cpp` at commit `3ba5fb3ca`, 9 July 2026.